Hybrid Critical-Bit 8T/Non-Critical-Bit 6T SRAM-Based Digital Computing-in-Memory with a Three-Mode BF16 Mantissa Multiplier

Maheswar Reddy Yarrabolu*, Anisetty Siva Ravi Teja*
*SENSE, Vellore Institute of Technology, Vijayawada, India
reddymaheshwar964@gmail.com; tejsiva95@gmail.com

***Abstract*—SRAM-based digital computing-in-memory (DCIM) can reduce the energy and latency associated with repeatedly moving data between memory and arithmetic units. A conventional design, however, often applies the same SRAM cell to every stored bit and uses a fixed arithmetic precision. This uniform organization may introduce unnecessary area and switching activity when different bits and workloads have different requirements. This paper presents a hybrid critical-bit 8T/non-critical-bit 6T SRAM-based DCIM architecture combined with a configurable three-mode BF16 mantissa multiplier. In the proposed organization, selected high-significance BF16 bits are stored in read-decoupled 8T cells, while less-significant bits use compact 6T cells. The arithmetic datapath supports 8×8, 6×6, and 4×4 mantissa multiplication modes, allowing the precision of the computation to be selected according to the operating requirement. The architecture was described in Verilog HDL and functionally verified at RTL. Logic synthesis was carried out using Cadence Genus, followed by physical implementation in Cadence Innovus through floorplanning, power planning, placement, clock-tree synthesis, and routing. Representative 6T and 8T SRAM cells were separately designed and simulated at transistor level in Cadence Virtuoso. The results demonstrate a complete design flow from the proposed architecture to a routed digital implementation, while also identifying physical verification as the remaining step toward final sign-off.**

***Index Terms*—SRAM, computing-in-memory, digital computing-in-memory, BF16, hybrid 8T/6T SRAM, configurable-precision multiplier**

I. Introduction

Deep neural-network accelerators execute a large number of multiply–accumulate operations. In a conventional memory-centric architecture, operands repeatedly move between the memory system and the arithmetic datapath. The associated data movement can consume a substantial portion of the overall energy and can also limit throughput. Digital computing-in-memory (DCIM) addresses this problem by moving selected computation closer to the stored data, reducing the distance between storage and arithmetic operations [2].

SRAM is an attractive memory technology for such architectures because of its fast access and compatibility with digital logic. The conventional 6T SRAM cell provides good density, but its read operation uses the storage nodes directly through the bit-lines. Read-decoupled 8T SRAM cells separate the read path from the storage nodes and can therefore provide improved read isolation, although this benefit comes with additional transistors and cell area. Selecting one cell type for an entire memory array consequently creates a system-level trade-off between density and read robustness.

This work follows a significance-aware approach instead of applying one cell type to every bit. BF16 contains one sign bit, eight exponent bits, and seven stored fraction bits. The exponent determines the scale of the represented value, while the fraction determines the precision of the significand. In the proposed organization, the exponent and selected upper fraction bits are treated as critical bits and assigned to 8T cells. The sign bit and lower fraction bits are assigned to compact 6T cells. The same principle is extended to the arithmetic datapath through a configurable three-mode mantissa multiplier.

The main contributions of this work are as follows:

- A hybrid 8T/6T SRAM organization that assigns cell type according to the significance of the stored BF16 bits.
- A shared BF16 mantissa multiplier with 8×8, 6×6, and 4×4 operating modes.
- A Verilog RTL implementation of the memory interface, controller, BF16 datapath, multiplier, and accumulation logic, followed by functional verification.
- Transistor-level implementation and simulation of representative 6T and 8T SRAM cells using Cadence Virtuoso.
- Synthesis and physical implementation of the digital architecture using Cadence Genus and Cadence Innovus through the routing stage.

II. Proposed Architecture

The proposed macro combines memory storage, memory-access control, BF16 arithmetic, multiplication, and accumulation in a single digital datapath. Fig. 1 shows the system-level organization used for the RTL implementation.

The principal blocks are the read/write controller, input interface, hybrid SRAM storage, BF16 computation unit, three-mode mantissa multiplier, accumulator, and output logic. The controller sequences memory access and computation. During a compute operation, one operand can be obtained from the stored BF16 word while the second operand is supplied through the input interface.

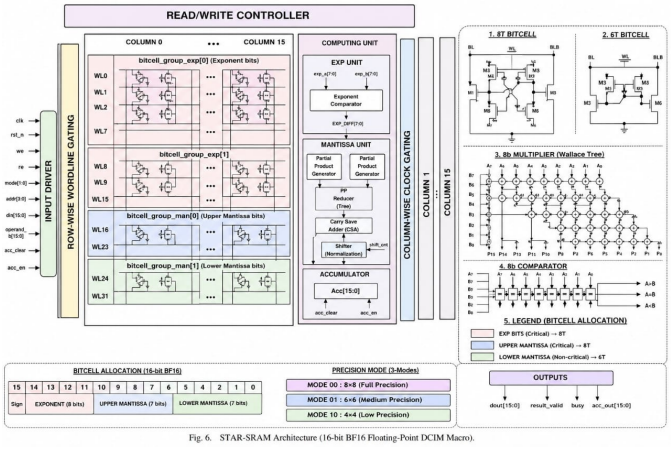


Fig. 6. STAR-SRAM Architecture (16-bit BF16 Floating-Point DCIM Macro).

Fig. 1. System-level organization of the proposed hybrid SRAM-based BF16 DCIM architecture.

A. BF16 Representation

BF16 is a reduced-precision floating-point format used in deep-learning workloads [1]. A BF16 word contains 16 bits arranged as

$$\mathrm{BF16} = \{S, E[7:0], F[6:0]\}, \tag{1}$$

where S is the sign bit, E is the eight-bit biased exponent, and F is the seven-bit stored fraction. For a normalized value, the arithmetic significand is written as $1.F$. The exponent uses a bias of 127.

For multiplication, the encoded exponents are added and the bias is subtracted once to obtain the resulting exponent. If the significand product exceeds the normalized range, the product is shifted and the exponent is incremented. The RTL datapath used in this study follows a compact implementation suitable for synthesis. Zero-valued exponent fields are treated as zero inputs, exceptional exponent patterns receive basic handling, and the fraction output is truncated to the required BF16 width.

B. Hybrid SRAM Organization

The proposed memory does not use the same SRAM cell for all 16 BF16 bits. The bit allocation used in this study is summarized in Table I. The exponent bits and upper fraction bits are assigned to 8T cells because an error in these positions can have a larger influence on the represented numerical value. The sign bit and lower fraction bits are assigned to 6T cells to retain the area and density advantages of the conventional SRAM cell.

This arrangement is intended to avoid the area penalty of using 8T cells throughout the array while retaining additional read isolation for the bits selected as critical. It therefore provides a middle ground between an all-6T implementation and an all-8T implementation. The exact benefit of this organization must ultimately be established through extracted-layout measurements and variation analysis.

TABLE I
PROPOSED BF16 BIT-TO-SRAM ALLOCATION

BF16 field	Cell	Design rationale
Exponent	8T	Improved read isolation for numerically important bits
Upper fraction	8T	Protection of higher-significance precision bits
Sign	6T	Compact storage for the sign information
Lower fraction	6T	Reduced cell-area and memory overhead

C. Three-Mode BF16 Mantissa Multiplier

A normalized BF16 significand contains an implicit leading one followed by the seven stored fraction bits. The proposed multiplier exposes three operating modes. The full-precision mode uses the complete eight-bit significands. The medium-precision mode retains the upper six fraction positions and suppresses the lower two. The low-power mode retains the upper four fraction positions and suppresses the lower three fraction positions.

The effective operands for the three modes can be represented as

$$M_8 = (1.F_A)(1.F_B), \tag{2}$$

$$M_6 = (1.F_{A[6:2]}00)(1.F_{B[6:2]}00), \tag{3}$$

and

$$M_4 = (1.F_{A[6:4]}000)(1.F_{B[6:4]}000). \tag{4}$$

Thus, the three modes correspond to 8×8, 6×6, and 4×4 effective mantissa multiplication. The RTL uses a common multiplier datapath with mode-dependent operand selection rather than duplicating three complete multipliers. Reducing the number of active operand bits can reduce unnecessary switching; however, the actual energy improvement depends on the implementation, input activity, and clock-gating or operand-isolation strategy.

III. RTL DESIGN AND FUNCTIONAL VERIFICATION

The digital architecture was described in Verilog HDL. The RTL was organized into modules for the memory model, three-mode mantissa multiplier, BF16 computation unit, accumulator, controller, and top-level DCIM macro. This modular organization allows the controller and arithmetic blocks to be verified independently before they are integrated at the top level.

The controller implements the sequencing required for reset, write, read, compute, wait, accumulate, and accumulator-clear operations. A command is accepted when the macro is idle, after which the controller generates the required memory and arithmetic control signals. The precision mode is supplied through a two-bit control input, allowing the three multiplier configurations to use the same top-level interface.

The RTL testbench applies reset, memory-write, memory-read, and compute transactions. It also exercises the available multiplier modes so that the control path and datapath can be

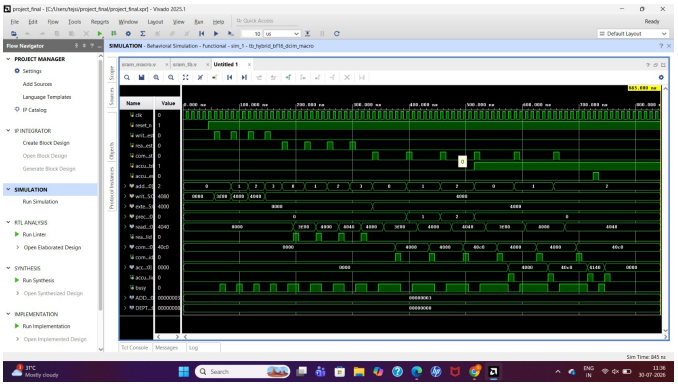


Fig. 2. RTL simulation evidence for the proposed BF16 DCIM datapath.

observed together. Fig. 2 shows the available RTL simulation evidence.

The RTL simulation confirms the intended functional sequence at the architectural level. The behavioral memory used for RTL verification is separate from the transistor-level SRAM cells described in the following section. This distinction is maintained so that circuit-level memory behavior is not overstated based only on behavioral RTL results.

IV. Transistor-Level SRAM Implementation

Representative 6T and 8T SRAM cells were implemented and simulated separately in Cadence Virtuoso. These cells provide the circuit-level basis for the hybrid memory organization. The present study focuses on demonstrating the cell structures and their simulation environments; complete extracted-layout characterization is left for future work.

A. 8T SRAM Cell

The 8T cell uses a read-decoupled path so that the read operation is separated from the internal storage nodes. This structure is useful for the proposed critical-bit locations because it reduces direct interaction between the storage nodes and the read bit-line during a read operation.

Fig. 3 shows the implemented 8T SRAM schematic, while Fig. 4 shows the corresponding simulation evidence.

The available simulation evidence demonstrates the operation of the read and storage nodes under the applied test conditions. Since complete numerical measurements of read delay, static noise margin, leakage, and dynamic power are not available in the recovered project material, these quantities are not assigned numerical values in this paper.

B. 6T SRAM Cell

The conventional 6T SRAM cell uses two cross-coupled inverters for data retention together with access transistors controlled by the word line. The complementary bit-lines are used for read and write operations. Its compact structure makes it suitable for the non-critical positions in the proposed hybrid organization.

Fig. 5 shows the 6T SRAM test environment, and Fig. 6 shows the associated transient simulation evidence.

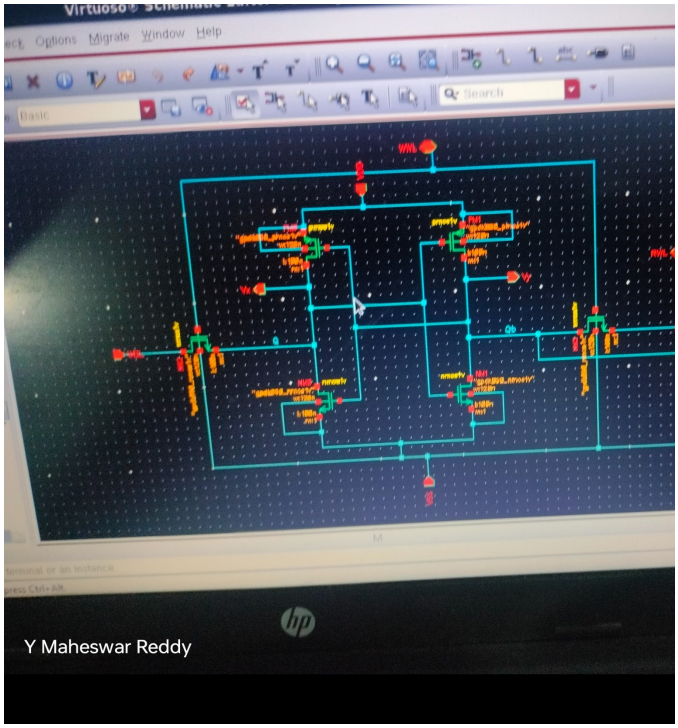


Fig. 3. 8T SRAM cell implemented at transistor level in Cadence Virtuoso.

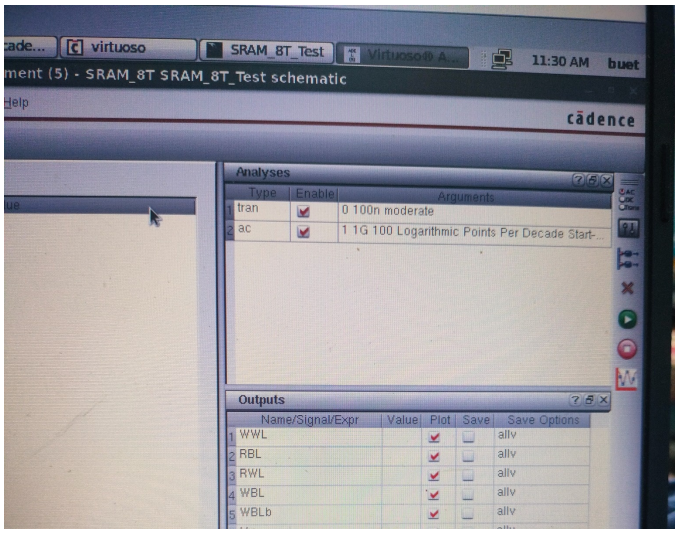


Fig. 4. Transient simulation evidence for the implemented 8T SRAM cell.

The separate implementation of the two cells allows the proposed architecture to be studied at both the memory-circuit and system levels. In a complete macro implementation, the selected 8T and 6T cells would be mapped to their corresponding BF16 bit positions according to the allocation described in Table I.

V. Logic Synthesis and Physical Implementation

The digital portion of the proposed architecture was taken through an RTL-to-place-and-route flow. Cadence Genus was used for logic synthesis, and Cadence Innovus was used

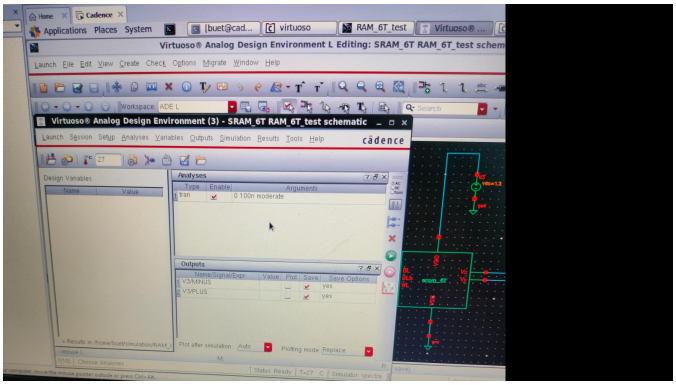


Fig. 5. 6T SRAM transistor-level test environment in Cadence Virtuoso.

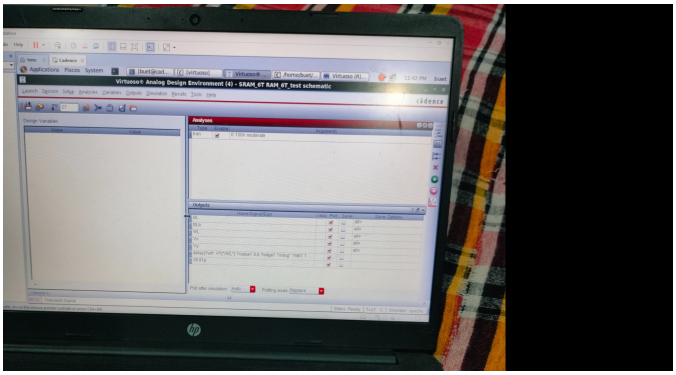


Fig. 6. Transient simulation evidence for the implemented 6T SRAM cell.

for backend implementation. The flow included synthesis, floorplanning, power planning, standard-cell placement, clock-tree synthesis, and detailed routing.

A. Logic Synthesis

The Verilog design was elaborated and synthesized to obtain a timing-constrained gate-level implementation. The reported top-level module was `hybrid_bf16_dcim_macro`. The synthesis report lists 1562 cells and a total reported cell area of 11933.285 in the technology-library area units. The selected timing configuration produced a reported 0 ps slack path.

These values are implementation results for the selected synthesis constraints and technology library. They should not be interpreted as universal area or timing characteristics of the architecture because both quantities depend on the library, constraints, operating conditions, and synthesis settings.

B. Floorplanning and Power Planning

The synthesized netlist was imported into Cadence Innovus and a floorplan was established for the digital design. Power planning was then performed to create the required power and ground structure before standard-cell placement. This stage prepares the design for clock and signal routing while maintaining the intended supply connectivity.



Fig. 7. Cadence Innovus routing view of the implemented digital design.

C. Placement and Clock-Tree Synthesis

Standard cells were placed within the defined floorplan, followed by clock-tree synthesis. The available implementation report indicates a clock tree containing three reported clock buffers for the analyzed clock network. CTS was performed before detailed routing so that the clock distribution could be incorporated into the final interconnect structure.

D. Routing

Detailed routing was performed after placement and CTS to establish the signal and clock connections. Fig. 7 shows the available Innovus routing evidence.

The resulting implementation demonstrates that the proposed RTL architecture can be carried through synthesis and physical implementation to the routing stage. Final DRC and LVS sign-off are not claimed because the required physical-verification environment was not available for the completed run.

VI. Results and Discussion

The work was evaluated at three abstraction levels: architectural functionality, transistor-level SRAM implementation, and digital physical implementation. Keeping these levels separate is important because the RTL memory model, transistor-level SRAM cells, and synthesized digital logic provide different types of evidence.

At the architectural level, RTL simulation was used to verify the intended read, write, compute, and accumulation sequence. The three precision modes share the same multiplier interface, so the arithmetic precision can be changed through the mode control without creating three independent top-level datapaths.

At the circuit level, both 6T and 8T SRAM cells were implemented in Cadence Virtuoso. The available evidence contains transient simulations for both cells and a separate simulation environment for the 8T read path. These results support the circuit-level implementation of the two cell types,

TABLE II
IMPLEMENTATION SUMMARY

Item	Implementation result
BF16 format	1 sign + 8 exponent + 7 fraction bits
SRAM organization	Hybrid 8T/6T
Multiplier modes	8×8, 6×6, 4×4
Technology	90-nm GPDK090
RTL	Verilog HDL
RTL verification	Functional simulation completed
Synthesis	Cadence Genus
Reported cell count	1562
Reported cell area	11933.285 library area units
Physical implementation	Cadence Innovus through routing
Clock-tree evidence	3 reported clock buffers
Final DRC/LVS sign-off	Not completed in the available environment

but they do not by themselves establish final macro-level power, delay, or stability values.

At the physical-design level, synthesis produced a gate-level implementation with 1562 reported cells and a reported cell area of 11933.285 technology-library area units. The selected timing analysis reported a 0 ps slack path. The backend flow then progressed through floorplanning, power planning, placement, clock-tree synthesis, and routing. These results demonstrate implementation progress from RTL to routed digital design.

The principal architectural trade-off is between cell density and read robustness. An all-8T memory would introduce additional storage overhead, whereas an all-6T organization would maximize compactness but would not provide the same read-path isolation. The hybrid approach places the larger cell only at the bit positions selected as critical. In parallel, the three multiplier modes provide a way to adapt arithmetic precision to the workload. The expected energy benefit of lower precision should be verified using post-synthesis and post-layout measurements rather than assumed from the RTL structure alone.

VII. LIMITATIONS AND FUTURE WORK

The current implementation establishes the proposed architecture, transistor-level SRAM structures, RTL functionality, synthesis, and place-and-route stages. The main remaining step is complete physical verification of the implementation and characterization of the memory cells after layout extraction.

Future work will focus on four areas. First, extracted 6T and 8T SRAM simulations should be used to obtain quantitative read/write delay, static noise margin, leakage power, and dynamic power. Second, the hybrid array should be evaluated against all-6T and all-8T organizations under the same area and operating conditions. Third, explicit operand isolation or mode-dependent gating can be added to the multiplier so that the energy impact of the 8×8, 6×6, and 4×4 modes can be measured at gate and layout level. Finally, process, voltage, and temperature analysis should be performed to determine whether the significance-aware bit allocation remains effective under circuit variation.

VIII. CONCLUSION

This paper presented a hybrid critical-bit 8T/non-critical-bit 6T SRAM-based DCIM architecture with a configurable three-mode BF16 mantissa multiplier. The proposed memory organization assigns 8T cells to selected high-significance BF16 bits and compact 6T cells to lower-significance bits. A shared multiplier supports 8×8, 6×6, and 4×4 effective mantissa multiplication, providing configurable arithmetic precision.

The digital architecture was implemented in Verilog HDL and functionally verified at RTL. Logic synthesis was performed using Cadence Genus, and the design was subsequently implemented in Cadence Innovus through routing. Representative 6T and 8T SRAM cells were separately designed and simulated at transistor level using Cadence Virtuoso. The reported synthesis and backend results establish a continuous implementation path from the proposed architecture to a routed digital design. At the same time, the study clearly separates completed implementation stages from the DRC/LVS and extracted-memory characterization required for final physical sign-off.

ACKNOWLEDGMENT

The authors acknowledge Dr. K. Srinivas, Vellore Institute of Technology, for academic supervision and guidance during the project. The authors also acknowledge the VLSI laboratory facilities used for Cadence-based design, simulation, synthesis, and physical implementation.

REFERENCES

[1] D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen, J. Yang, J. Park, A. Heinecke, E. Georganas, S. Srinivasan, A. Kundu, M. Smelyanskiy, B. Kaul, and P. Dubey, "A Study of BFLOAT16 for Deep Learning Training," arXiv:1905.12322, 2019.

[2] S. Yu, H. Jiang, S. Huang, X. Peng, and A. Lu, "Compute-in-Memory Chips for Deep Learning: Recent Trends and Prospects," *IEEE Circuits and Systems Magazine*, vol. 21, no. 3, pp. 31–56, 2021.

[3] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[4] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.

[5] B. Jacob, S. W. Ng, and D. T. Wang, *Memory Systems: Cache, DRAM, Disk*. Burlington, MA, USA: Morgan Kaufmann, 2008.